Q3 – Engineer a Vehicle Registration Number Validator

Normalisation and validation

Strip leading/trailing whitespace, remove spaces and hyphens, and uppercase the result. A valid registration has two letters, two digits, one or two series letters, and four digits; therefore total length is 9 or 10. Validate each slice with isalpha() and isdigit().

String concepts

Strings are immutable, so upper() returns a new string and leaves the original unchanged. For a 9-character registration, s[-4:] returns the final four digits and s[4:-4] returns one series letter. For a 10-character registration, the same slices return four digits and two series letters.

Alternative regular expression

re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', reg) expresses the grammar compactly. Both methods are O(L) in input length; slicing is transparent for this fixed format, while a regular expression is easier to extend for more complex patterns.

Test evidence

The supplied samples normalise to TN09AB1234 (valid), KA01M0007 (valid), TN9AB1234 (invalid) and TN221234AB (invalid). Receipt formatting groups the validated slices.

Python Program

```
def normalise(text):
    return text.strip().replace(" ", "").replace("-", "").upper()

def is_valid(reg):
    if len(reg) not in (9, 10):
        return False
    series_end = len(reg) - 4
    return (reg[:2].isalpha() and reg[2:4].isdigit()
            and reg[4:series_end].isalpha()
            and len(reg[4:series_end]) in (1, 2)
            and reg[series_end:].isdigit())

def format_receipt(reg):
    if not is_valid(reg):
        raise ValueError("Invalid registration number")
    return f"{reg[:2]} {reg[2:4]} {reg[4:-4]} {reg[-4:]}"

if __name__ == "__main__":
    samples = ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]
    for raw in samples:
        value = normalise(raw)
        print(raw, "->", value, "valid" if is_valid(value) else "INVALID")
```